

Ros2 최종프로젝트 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 노드 구성
3. 코드 구조
4. 파일 구조
5. 실행결과

1. 프로젝트 개요

이 프로젝트는 ROS2와 Qt를 사용해 만든 길건너 친구들 게임이다. 여러 차선을 달리는 차를 피해서 캐릭터를 반대편까지 옮기는 것이 목표다.

ROS2 교육 3일차에 만든 신호등 교차로 시뮬레이터를 바탕으로 시작했다. 원래는 차가 신호에 따라 달리고 멈추는 것을 Qt로 보여주기만 하는 프로그램이었는데, 여기에 플레이어를 추가하고 충돌 판정과 점수를 붙여서 게임으로 만들었다.

역할을 나눌 때 기준으로 삼은 것은 Qt가 로직을 갖지 않게 하는 것이다. 신호를 바꾸는 것, 차를 움직이는 것, 충돌을 판정하는 것은 전부 노드 쪽에서 하고, Qt는 토픽으로 받은 결과를 화면에 그리고 버튼 입력을 발행하는 일만 한다. 그래서 Qt를 끄고 터미널에서 토픽만 확인해도 게임 자체는 계속 돌아간다.

설정값은 소스 코드에 넣지 않고 yaml 파일로 뺐다. 차선 수, 차선별 속도, 신호 점등 시간, 차 생성 주기 같은 것들을 코드 수정 없이 바꿀 수 있어서 난이도를 맞추는 데 편했다.

2. 노드 구성

노드는 총 다섯개 이다.

traffic_light_node	신호등 상태를 RED -> GREEN -> YELLOW 순으로 순환시키고 발행
vehicle_node	차를 생성하고 이동시키며 전체 차량 위치를 발행
player_node	플레이어 위치 관리, 충돌 판정, 점수 계산
game_node	리셋 요청을 받아 시작 위치를 돌려주는 서비스 서버
road_friend_gui	Qt 화면. 구독한 값을 그리고 버튼 입력을 발행

토픽은 총 4개이다.

```
traffic_light_node —/traffic_light—> vehicle_node
vehicle_node —/vehicle_pos—┐——> player_node
                           └——> road_friend_gui
road_friend_gui —/player_cmd—> player_node
player_node —/player_state—┐——> road_friend_gui
                           └——> vehicle_node
player_node —[reset_game 서비스]——> game_node
```

/vehicle_pos와 /player_state는 구독자가 둘씩이다. 같은 데이터를 화면에도 쓰고 로직에도 쓰기 때문인데, 토픽이 여러 노드에 동시에 전달되는 구조라 발행하는 쪽은 누가 받는지 신경 쓰지 않아도 된다.

서비스는 리셋에 사용했다. 플레이어가 차에 부딪히면 player_node가 game_node에 요청을 보내고, game_node가 시작 위치를 응답으로 돌려준다. 요청하고 답을 받아야 하는 동작이

라 토픽 대신 서비스를 썼다.

메세지는 4개이고 서비스는 한 개이다.

```
# Car.msg
```

```
int32 lane
```

```
float32 x
```

```
# CarArray.msg
```

```
Car[] cars
```

```
# Move.msg
```

```
int32 dx
```

```
int32 dy
```

```
# PlayerState.msg
```

```
int32 row
```

```
int32 col
```

```
bool alive
```

```
int32 score
```

```
# Reset.srv
```

```
int32 deaths
```

```
---
```

```
int32 start_row
```

```
int32 start_col
```

차가 여러 대라서 배열을 담을 CarArray가 따로 필요했다. 토픽은 메시지 하나만 실을 수 있어서 배열을 감싸는 타입을 하나 더 만들어야 했다.

3. 코드 구조

신호등 노드

```
void TrafficLightNode::timer_callback()
```

```
{
```

```
    elapsed_ += 0.1;
```

```
    if (state_ == "RED" && elapsed_ >= r_time_)
```

```

{
    state_ = "GREEN";
    elapsed_ = 0;
}
...
std_msgs::msg::String msg;
msg.data = state_;
publisher_ -> publish(msg);
}

```

0.1초마다 실행되면서 경과 시간을 누적하고, 현재 상태의 제한 시간을 넘으면 다음 상태로 넘어간다. 상태를 바꾸는 것과 별개로 발행은 매번 한다. 그래야 Qt가 늦게 켜져도 바로 현재 신호를 받을 수 있다고 생각하기 때문이다.

상태 전환마다 타이머를 새로 만드는 방법도 있지만, 그렇게 하면 나중에 들어온 구독자가 다음 전환이 올 때까지 아무것도 못 받는다.

차량 노드/앞차 간격 유지용

```

std::vector<float> front(lane_count_, 999999.0);
for (size_t i = 0; i < vehicles_.size(); i++)
{
    int lane = vehicles_[i].lane;
    double next = vehicles_[i].x + v * dt;

    if (next > front[lane] - 40.0)
    {
        next = front[lane] - 40.0;
    }

    vehicles_[i].x = next;
    front[lane] = next;
}

```

차선이 하나였을 때는 배열에서 바로 앞 인덱스가 앞차였다. 차선이 여러 개가 되면서 배열에 여러 차선이 섞이게 되어 그 방법을 쓸 수 없게 됐다.

그래서 차선 개수만큼 칸을 가진 front 배열을 두고, 순회하면서 각 차선의 직전 차 위치를 기록해두는 방식으로 바꿨다. 배열이 생성 순서대로 정렬되어 있어서 같은 차선 안에서는 먼저 생긴 차가 항상 앞에 있고, 따라서 직전에 처리한 같은 차선 차가 곧 앞차가 된다.

초기값을 999999로 둔 것은 앞차가 없다는 뜻이다. 0으로 두면 그 차선의 첫 차가 -40으로 밀려서 화면 밖으로 나가버린다.

이 방식은 차를 한 번만 훑으면 끝나서, 모든 차끼리 거리를 비교하는 것보다 계산이 적다.

차량노드/정지선, 점수 연동

```
if (light_ == "RED" && vehicles_[i].x <= stop_line_)
```

```
{
    if (next > stop_line_)
    {
        next = stop_line_;
    }
}
```

빨간불일 때 차를 세우는 부분이다. 조건에 vehicles_[i].x <= stop_line_이 들어간 것은 이미 정지선을 지난 차는 계속 가야 하기 때문이다. 신호만 보고 멈추게 하면 교차로 한가운데 차가 서버린다.

```
player_sub_ = this->create_subscription<road_friend::msg::PlayerState>(
```

```
    "player_state", 10,
    [this](const road_friend::msg::PlayerState::SharedPtr msg)
    {
        speed_scale_ = 1.0 + msg->score * speed_step_;
    });
```

차량 노드가 플레이어 상태를 구독해서 점수만큼 속도 배율을 올린다. 점수가 오를수록 차가 빨라져서 난이도가 올라간다. 배율 증가폭도 yaml로 빼두어 조절할 수 있게 했다.

플레이어노드/충돌판정용

```
if (row_ < 1 || row_ > 5)
```

```
{
    return;    // 안전지대
}
```

```
int lane = 5 - row_;
```

```
double px = col_ * col_width_;
```

```
for (size_t i = 0; i < msg->cars.size(); i++)
```

```
{
    if (msg->cars[i].lane != lane)
    {
```

```

        continue;
    }

    double cx = msg->cars[i].x;
    if (px + player_width_ > cx && px < cx + car_width_)
    {
        alive_ = false;
        ...
    }
}

```

먼저 플레이어가 차선 위에 있는지 확인하고, 아니면 판정 자체를 건너뛴다. 출발지와 도착지에서는 차에 맞을 일이 없기 때문이다.

5 - row_는 행 번호를 차선 번호로 바꾸는 부분이다. 행은 아래에서 위로 세고 차선은 위에서 아래로 세기 때문에 순서가 뒤집힌다.

겹침 판정은 두 구간이 겹치는지를 보는 것이다. 플레이어 구간은 $px \sim px + player_width_$, 차 구간은 $cx \sim cx + car_width_$ 이고, 서로의 시작점이 상대의 끝점보다 앞에 있으면 겹친 것이다. 가로 방향만 보면 되는 이유는 같은 차선인지를 이미 걸렀기 때문이다.

플레이어노드/서비스요청

```

void PlayerNode::request_reset()
{
    if (!reset_client_->wait_for_service(std::chrono::milliseconds(100)))
    {
        RCLCPP_WARN(this->get_logger(), "reset service not available");
        return;
    }

    auto req = std::make_shared<road_friend::srv::Reset::Request>();
    req->deaths = deaths_;

    reset_client_->async_send_request(req,
        [this](rclcpp::Client<road_friend::srv::Reset>::SharedFuture future)
        {
            auto res = future.get();

```

```

        row_ = res->start_row;
        col_ = res->start_col;
        alive_ = true;
        publish_state();
    });
}

```

서비스 서버가 아직 안 떠 있을 수 있어서 먼저 확인하고, 없으면 경고만 남기고 돌아간다. 응답을 기다리는 방식으로 호출하면 노드가 멈춰버린다. 특히 이 함수는 구독 콜백 안에서 호출되기 때문에, 응답을 기다리는 동안 스핀이 막혀서 응답을 영영 받지 못하는 상태가 된다. 그래서 요청을 보내고 바로 돌아온 뒤, 응답이 오면 콜백에서 처리하도록 했다.

예외처리

```

if (!alive_)
{
    return;
}

int next_row = row_ + msg->dy;
int next_col = col_ + msg->dx;

if (next_row < 0 || next_row > 6)
{
    return;
}
if (next_col < 0 || next_col >= col_count_)
{
    return;
}

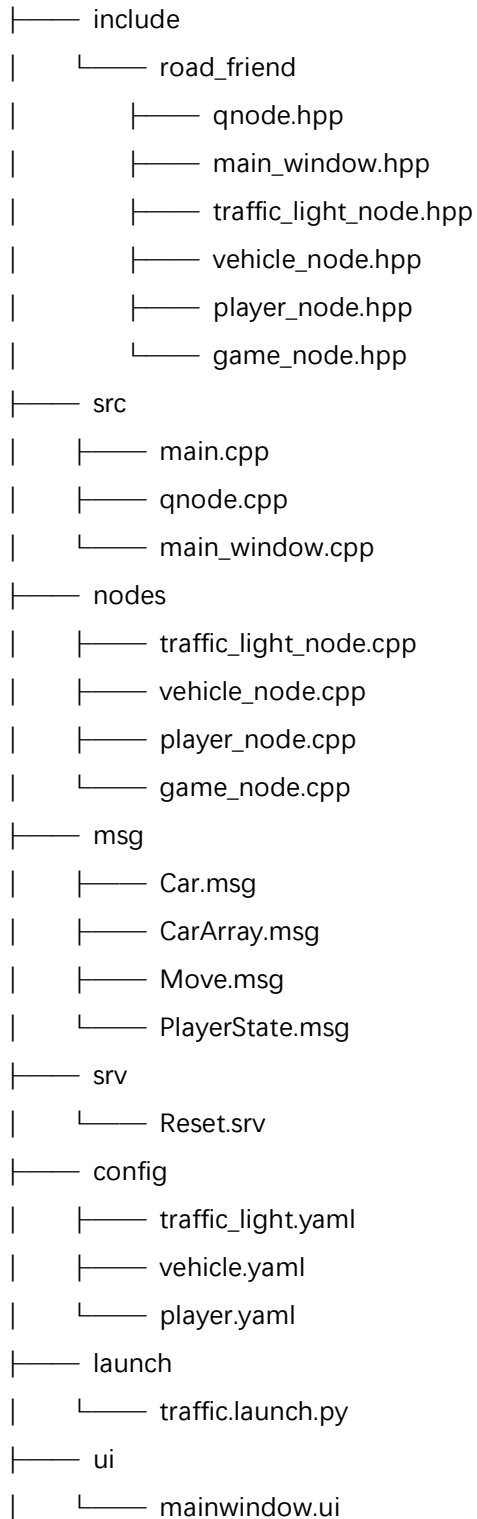
```

이동 요청이 들어오면 죽은 상태인지, 화면 밖으로 나가는지를 먼저 확인한다. 바로 row_에 더하지 않고 후보값을 만들어서 검사한 뒤 통과하면 반영하는 방식이라, 잘못된 값이 들어와도 원래 위치가 유지된다.

이 밖에도 서비스 가용성 확인, 안전지대에서 충돌 판정 건너뛰기, 차선 배열 범위 확인 등을 넣었다.

4. 파일 구조

road_friend

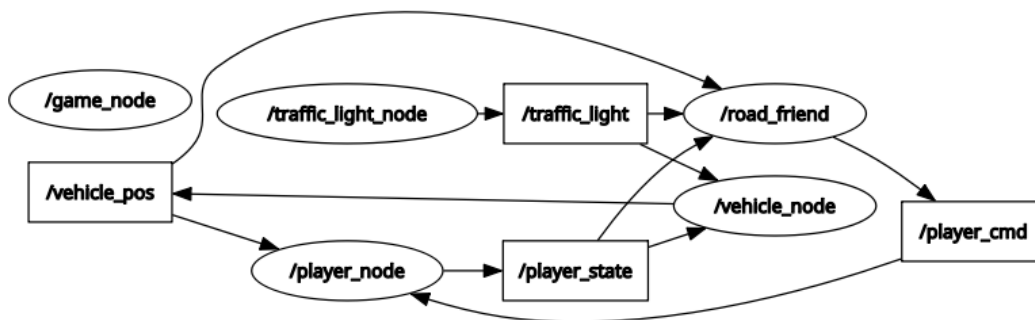


└── CMakeLists.txt

└── package.xml

헤더와 소스를 나누고, 헤더는 include/road_friend/ 아래에 두었다. 패키지 이름 폴더가 한번 더 들어가는 것은 다른 패키지에서 가져다 쓸 때 이름이 겹치지 않게 하기 위함이다.

5, 실행결과



traffic_light_node가 발행한 신호를 vehicle_node와 GUI가 함께 받고, vehicle_node가 발행한 차량 위치를 player_node와 GUI가 함께 받는다. GUI는 버튼 입력을 /player_cmd로 보내고, player_node가 그 결과를 /player_state로 돌려주는 구조라 양쪽으로 화살표가 이어진다. game_node는 토픽을 쓰지 않고 서비스만 제공하기 때문에 연결선 없이 따로 떨어져 있다.

(첨부 영상 참고)